



Documentación del Proceso de Desarrollo

ÍNDICE:

.....	1
1. Introducción.....	2
2. Escenario	3
3. Patrones de diseño aplicados y principios SOLID.....	4
4. Enemigos	6
5. IA DE LOS ENEMIGOS:	7
6. Sistema de monedas.....	8
7. WinZone, estados finales y bucle de juego	10
8. GameManager y Control del juego.....	11
9. Comparación entre el diseño inicial y el diseño final	11
10. Conclusiones finales	12
11. Uso de la IA:.....	12

En relación con la versión compilada del proyecto, se adjunta el ejecutable solicitado, aunque, debido a problemas técnicos surgidos en el proceso de compilación con el NavMesh, el comportamiento de los enemigos no se reproduce de forma completamente correcta fuera del entorno de Unity. Por este motivo, se ruega que la evaluación principal del funcionamiento del juego se realice a partir del proyecto ejecutado directamente en Unity, así como de los vídeos de demostración entregados, en los que se puede comprobar el comportamiento correcto de los enemigos, el sistema de navegación y el desarrollo completo de la partida tal y como fue diseñado e implementado.

1. Introducción

Este documento recoge de forma detallada y estructurada todo el proceso de desarrollo del proyecto final de la asignatura **Paradigmas de Programación**. El objetivo principal de esta memoria es reflejar fielmente el trabajo realizado, explicando no solo el resultado final, sino también el camino recorrido hasta alcanzarlo, incluyendo las decisiones tomadas, los errores cometidos, los problemas encontrados y las soluciones aplicadas en cada fase del desarrollo.

A lo largo del documento se compara el planteamiento inicial del proyecto con la versión final implementada, justificando de manera razonada cada uno de los cambios que se produjeron. Se pone especial énfasis en las dificultades reales surgidas durante el desarrollo del videojuego, ya que estas han sido una parte fundamental del aprendizaje. El proyecto no se desarrolló de forma lineal ni sin contratiempos, sino que requirió constantes ajustes, pruebas, refactorizaciones de código y replanteamientos de diseño para poder avanzar de forma estable.

El proyecto consiste en el desarrollo de un videojuego en **Unity**, en el que el jugador debe desplazarse por el escenario evitando a los enemigos controlados por inteligencia artificial y cumpliendo una serie de objetivos para poder ganar la partida. Aunque inicialmente el juego estaba ambientado en un laboratorio futurista, durante el desarrollo fue necesario cambiar el escenario a un castillo medieval debido a limitaciones técnicas y de recursos. Este cambio tuvo un impacto directo en la estructura del juego y en la implementación de sistemas clave como la navegación, la detección del jugador y el movimiento de los enemigos.

Durante el desarrollo se aplicaron distintos paradigmas de programación, destacando principalmente la **programación orientada a objetos**, la **gestión de estados** tanto a nivel de juego como de los enemigos, y la implementación de **algoritmos de comportamiento** para la inteligencia artificial. Todo ello permitió construir una base sólida y coherente, en la que cada sistema depende y se comunica correctamente con los demás.

2. Escenario

Inicialmente, la idea era ambientar el juego en un laboratorio, ya que encajaba bien con el concepto original. Sin embargo, durante la búsqueda de recursos, no se encontró ningún escenario de laboratorio que cumpliera con los requisitos mínimos del proyecto: que fuese lo suficientemente grande, visualmente claro, bien estructurado y, sobre todo, compatible con el sistema de navegación y la lógica de enemigos que se estaba implementando. Muchos de los laboratorios disponibles eran demasiado pequeños, estaban excesivamente cerrados o presentaban una geometría poco adecuada para trabajar correctamente.

Ante estas limitaciones, se optó por cambiar a un castillo medieval, un entorno que sí ofrecía un espacio amplio, variado y visualmente reconocible. Además, este escenario permitía una mayor riqueza espacial, con zonas abiertas, patios, pasillos estrechos y diferentes alturas, lo que aportaba más posibilidades jugables y encajaba mejor con la mecánica de exploración y persecución del juego. No obstante, este cambio supuso rehacer parte del planteamiento

inicial, ya que el laboratorio estaba concebido como un entorno cerrado y controlado, mientras que el castillo introduce una estructura mucho más irregular y compleja.

3. Patrones de diseño aplicados y principios SOLID

A lo largo del desarrollo del proyecto se aplicaron distintos patrones de diseño con el objetivo de mantener una estructura de código clara, modular y fácil de mantener. Estas decisiones no se tomaron de forma teórica, sino como respuesta directa a los problemas que fueron surgiendo durante la implementación del videojuego y a la necesidad de adaptar el proyecto a cambios constantes, como el cambio de escenario, el aumento de la complejidad de la inteligencia artificial y la introducción de distintos niveles de dificultad.

Uno de los patrones más relevantes utilizados en el proyecto es el **patrón Singleton**, implementado a través del Gamemanager. Este patrón garantiza que exista una única instancia encargada de gestionar el estado global del juego, lo que resulta fundamental en un videojuego donde múltiples sistemas deben estar sincronizados. El Gamemanager centraliza el control de los estados del juego (menú, partida en curso, victoria y derrota), la gestión de la interfaz de usuario, el conteo de monedas recogidas, la activación de alertas globales y las condiciones de victoria. Gracias a este enfoque, se evitó la duplicación de lógica y la aparición de inconsistencias, ya que todos los sistemas consultan y modifican un único punto de referencia. En un proyecto de este tamaño, el uso del Singleton está plenamente justificado, ya que simplifica la comunicación entre componentes y reduce el acoplamiento innecesario entre scripts.

Otro patrón clave en el desarrollo del juego es el **patrón State**, aplicado tanto a nivel global del juego como al comportamiento de los enemigos. El uso de una máquina de estados permitió definir de forma clara cómo debe comportarse el sistema en cada momento y bajo qué condiciones se producen las transiciones entre estados. En el caso del juego, esto se traduce en una separación clara entre los estados de menú, juego activo, victoria y derrota. En el caso de los enemigos, los estados permiten diferenciar el comportamiento de patrulla, detección, persecución y retorno a la ruta de waypoints. Este enfoque fue especialmente importante para evitar comportamientos erráticos, como que un enemigo se quedara fijado en un waypoint cuando el jugador se encontraba muy cerca, ya que el cambio de estado a persecución fuerza al enemigo a dirigirse directamente hacia el jugador. El patrón State aportó orden, claridad y previsibilidad al comportamiento del juego y de la inteligencia artificial.

El **patrón Factory** también tuvo un papel fundamental en el proyecto. Se utilizó una factoría o sistema de creación centralizado para instanciar tanto a los enemigos como a las monedas del juego. Esta decisión fue especialmente importante debido a la existencia de tres niveles de dificultad (fácil, normal y difícil), cada uno con un número distinto de monedas y enemigos. Centralizar la creación de estos elementos permitió que la lógica de dificultad estuviera concentrada en un único punto del sistema, evitando tener que modificar manualmente la escena o múltiples scripts cada vez que se ajustaba el equilibrio del juego. De este modo, la factoría se encarga de decidir cuántos enemigos aparecen, de qué tipo son y cuántas monedas se generan en función de la dificultad seleccionada. Este enfoque facilita enormemente la

escalabilidad del proyecto y reduce errores de configuración, además de permitir que el diseño de niveles sea más flexible y reutilizable.

En relación con la inteligencia artificial, el proyecto aplica de forma implícita el **patrón Strategy**, ya que los enemigos pueden presentar comportamientos distintos en función de su tipo, como exploradores o guardianes. Aunque comparten una base común de movimiento, navegación y detección, cada tipo de enemigo introduce variaciones en su comportamiento, como la activación de una alerta global durante un tiempo determinado. Esta separación de comportamientos evita la duplicación de código y permite ampliar el sistema de enemigos en el futuro sin necesidad de reescribir la lógica principal de la IA. El uso de este patrón permitió mantener una base sólida y flexible para el comportamiento de los enemigos.

Además, el proyecto sigue un enfoque cercano al **patrón Observer**, ya que muchos sistemas reaccionan a eventos que ocurren durante la partida. La recogida de monedas, la detección del jugador por parte de un enemigo o la activación de la alerta global son ejemplos de sucesos que desencadenan cambios en otros sistemas, como la interfaz de usuario o el estado del juego. Aunque estas comunicaciones se realizan principalmente a través del Gamemanager, el concepto subyacente es el de un sistema basado en eventos, donde un cambio en una parte del juego produce efectos controlados en otras, sin necesidad de un acoplamiento directo excesivo.

En conjunto, la arquitectura del proyecto respeta de forma clara los **principios SOLID**, lo que contribuye a la calidad y mantenibilidad del código. El principio de **responsabilidad única** se cumple al asignar a cada clase una función concreta, como la gestión del estado global, el control de la IA de los enemigos, el cálculo de rutas mediante A* o la gestión de la recogida de monedas. El principio de **abierto/cerrado** se refleja en la posibilidad de ampliar el juego con nuevos tipos de enemigos, nuevas reglas de dificultad o modificaciones del escenario sin necesidad de reescribir el núcleo del sistema. El principio de **sustitución de Liskov** se mantiene al garantizar que los distintos tipos de enemigos respetan el comportamiento esperado dentro del sistema común de IA. El principio de **segregación de interfaces** se aplica de forma estructural al separar claramente los sistemas de navegación, comportamiento, interfaz y lógica global. Por último, el principio de **inversión de dependencias** se ve reflejado en la forma en que los sistemas dependen de estructuras y reglas generales, como el grafo de navegación o el gestor global del juego, en lugar de depender directamente de detalles concretos del escenario.

En conclusión, la aplicación de estos patrones de diseño y principios SOLID no solo mejoró la organización interna del proyecto, sino que fue clave para poder afrontar los cambios, problemas y ajustes que surgieron durante el desarrollo. Gracias a esta arquitectura, el proyecto pudo evolucionar desde una idea inicial sencilla hasta un sistema más complejo y robusto, manteniendo en todo momento un código comprensible, reutilizable y coherente con los objetivos del juego.

4. Enemigos

La lógica de cambio de estado de los enemigos se diseñó para que su comportamiento fuese **coherente, reactivo y acorde a la situación del jugador**, evitando transiciones artificiales o poco realistas. Para ello, cada enemigo funciona mediante una **máquina de estados** claramente definida, en la que el estado actual determina tanto su objetivo como la forma en la que se calcula su movimiento mediante el algoritmo A*.

En condiciones normales, los enemigos se encuentran en **estado de patrulla**. En este estado, su objetivo no es el jugador, sino una serie de puntos del NavGraph que actúan como waypoints. El enemigo calcula una ruta con A* hacia el siguiente waypoint y lo sigue hasta alcanzarlo. Una vez llega, selecciona el siguiente punto de patrulla y vuelve a calcular el camino. Este comportamiento asegura un movimiento constante y predecible por el escenario, sin quedarse estático ni realizar desplazamientos aleatorios.

El cambio de estado se produce cuando el enemigo **detecta al jugador**. La detección no se basa únicamente en la distancia, sino en una combinación de distancia máxima y comprobación de visibilidad. Si el jugador se encuentra dentro de ese rango de detección y no existen obstáculos que bloqueen la visión, el enemigo abandona inmediatamente el estado de patrulla. En ese momento, el objetivo deja de ser el waypoint más cercano y pasa a ser directamente la posición del jugador.

Cuando la detección se produce a una **distancia pequeña**, el cambio de comportamiento es inmediato. El enemigo no finaliza su ruta actual ni intenta volver al waypoint más cercano. En su lugar, se fuerza un **cambio de estado directo a persecución**, lo que provoca que el algoritmo A* se recalcule usando como nodo objetivo el nodo del NavGraph más cercano al jugador. Esto garantiza que el enemigo reaccione de forma instantánea y natural, transmitiendo sensación de peligro real al jugador.

En estado de persecución, el enemigo recalcula su ruta siempre que la posición del jugador cambia de forma significativa. De este modo, el enemigo no sigue un camino obsoleto ni se queda bloqueado intentando alcanzar un punto antiguo. Mientras el jugador se mantenga dentro del rango de detección, el enemigo continuará persiguiéndolo de forma activa, ignorando por completo los waypoints de patrulla.

Si el jugador deja de estar visible o se aleja más allá de la distancia establecida, el enemigo no vuelve inmediatamente a patrulla. En su lugar, permanece durante un breve intervalo en un estado intermedio de alerta, manteniendo la última ruta calculada o desplazándose hacia la última posición conocida del jugador. Esto evita cambios bruscos de comportamiento y hace que la IA resulte más creíble.

Uno de los problemas más persistentes durante el desarrollo fue que algunos enemigos quedaban estancados en determinadas zonas del escenario, especialmente cerca de paredes, esquinas o pasos estrechos. Este comportamiento se debía a una combinación de factores, como rutas calculadas a través de nodos mal posicionados, desajustes entre el NavGraph y las zonas realmente transitables del NavMesh, y cambios bruscos de objetivo provocados por la detección del jugador. Para solucionar este problema, se revisó cuidadosamente la

colocación y separación de los nodos del grafo de navegación, asegurando que todos estuvieran situados sobre superficies caminables y suficientemente alejados de colisionadores. Además, se ajustó la lógica de movimiento para permitir que los enemigos pudieran rotar y reorientarse correctamente durante el desplazamiento, evitando bloqueos causados por giros demasiado bruscos o direcciones imposibles. Gracias a estas correcciones, se consiguió un movimiento más fluido y estable, eliminando los bloqueos y mejorando de forma notable la credibilidad del comportamiento de la inteligencia artificial.

5. IA DE LOS ENEMIGOS:

Durante el desarrollo del proyecto, uno de los aspectos más importantes y complejos fue la implementación de la inteligencia artificial de los enemigos, ya que esta debía integrarse correctamente con el diseño del escenario y con las mecánicas principales del juego. Desde el inicio se descartó una solución excesivamente simple, ya que el mapa presenta una estructura compleja con murallas, edificios y zonas estrechas, lo que hacía imprescindible un sistema de navegación controlado y fiable.

En una primera fase, se detectó que el uso exclusivo de soluciones automáticas de navegación no ofrecía el control necesario sobre las rutas de los enemigos. En muchas situaciones, estos atravesaban muros, tomaban atajos irreales o quedaban atascados en zonas donde el terreno no estaba correctamente conectado. Por este motivo, se decidió implementar un sistema propio de pathfinding basado en el algoritmo A*, apoyado en una estructura de datos tipo NavGraph que representase de forma explícita las zonas transitables del escenario.

El NavGraph se construyó como un conjunto de nodos distribuidos estratégicamente por el mapa, situados únicamente en zonas realmente navegables. Cada nodo representa un punto válido de desplazamiento y se conecta con otros nodos cercanos siempre que no exista ningún obstáculo físico entre ellos. Esta aproximación permitió definir rutas claras alrededor de edificios y murallas, evitando comportamientos incorrectos y garantizando que los enemigos respetasen la geometría del escenario.

Una vez definido el NavGraph, el algoritmo A* se utilizó para calcular las rutas óptimas entre dos puntos: la posición actual del enemigo y su objetivo, que puede ser un waypoint de patrulla o la posición del jugador. El algoritmo evalúa los nodos disponibles, calcula el coste del camino recorrido y estima el coste restante hasta el objetivo, devolviendo finalmente una secuencia ordenada de nodos que el enemigo debe seguir. Esta ruta se traduce en movimiento real dentro del juego, permitiendo un desplazamiento coherente y controlado.

La navegación mediante A* se integró directamente en la lógica de la inteligencia artificial de los enemigos. En estado de patrulla, los enemigos calculan rutas entre distintos puntos del mapa para recorrer el escenario de forma predecible. Cuando el jugador es detectado, el objetivo cambia y se recalcula el camino hacia su posición, iniciando la persecución. Este recalculado no se realiza de forma constante, sino únicamente cuando se producen cambios relevantes, evitando así problemas de rendimiento y movimientos erráticos.

Durante la implementación de este sistema surgieron numerosos problemas que influyeron directamente en el diseño final. En primer lugar, algunos nodos del NavGraph estaban mal

posicionados, demasiado cerca de paredes o incluso dentro de colisionadores, lo que provocaba rutas imposibles o enemigos bloqueados. También se detectaron conexiones incorrectas entre nodos que atravesaban obstáculos, lo que obligó a revisar manualmente la estructura del grafo para asegurar su coherencia.

Otro problema frecuente fue que, en determinadas situaciones, el algoritmo A* devolvía una ruta válida, pero el enemigo no llegaba a desplazarse correctamente. Esto se debió a errores en la conversión de los nodos del grafo a movimiento real dentro del escenario, especialmente en zonas estrechas o con cambios de altura. Estos fallos obligaron a ajustar tanto la separación entre nodos como la lógica de avance entre ellos.

Además, se produjeron desajustes entre el NavGraph y el NavMesh del escenario, ya que no siempre coincidían exactamente las zonas consideradas transitables. Esta falta de sincronización provocó pequeños atascos y comportamientos inconsistentes, que se resolvieron refinando la colocación de nodos y asegurando que siempre estuvieran situados sobre superficies realmente caminables.

En conclusión, la implementación del algoritmo A* junto con un NavGraph propio fue una decisión clave para el correcto funcionamiento de la inteligencia artificial de los enemigos. Aunque supuso una carga importante de trabajo y numerosos problemas durante el desarrollo, permitió obtener un control preciso sobre las rutas, evitar comportamientos erráticos y adaptar la navegación a un escenario complejo. El resultado final es una IA coherente, robusta y totalmente integrada con el diseño del juego, que aporta desafío y credibilidad al comportamiento de los enemigos.

6. Sistema de monedas

Desde las primeras fases del desarrollo, el sistema de monedas no formaba parte del diseño inicial del juego. En el planteamiento original, el jugador podía desplazarse directamente hasta la zona de victoria (WinZone) sin necesidad de cumplir ningún requisito adicional, lo que hacía que la experiencia resultase excesivamente sencilla y poco desafiante. Tras varias pruebas, se detectó que esta mecánica eliminaba cualquier sensación de progresión y reducía drásticamente la duración y el interés de la partida. Por este motivo, se decidió introducir un sistema de monedas como requisito obligatorio para poder ganar, con el objetivo de forzar al jugador a explorar el escenario, recorrer zonas peligrosas y exponerse durante más tiempo a los enemigos.

El sistema de monedas se integró como una mecánica central de progreso y no como un elemento opcional. De este modo, la recogida de monedas condiciona directamente el ritmo de la partida y añade un componente estratégico, ya que el jugador debe decidir por dónde moverse, qué rutas tomar y cuándo asumir riesgos para completar el objetivo final. Esta decisión permitió transformar un juego inicialmente lineal en una experiencia más tensa y planificada.

La cantidad total de monedas que aparecen en cada partida se define en función del nivel de dificultad seleccionado. En el modo fácil se generan tres monedas, en el modo normal cuatro y en el modo difícil cinco. Esta diferencia se estableció para equilibrar la duración de la partida

y el nivel de exposición del jugador a los enemigos, ya que a mayor dificultad no solo aumenta el reto por la presencia de enemigos, sino también el tiempo necesario para completar el objetivo. De esta forma, la dificultad afecta tanto al comportamiento de la IA como a la estructura del propio nivel.

Toda la lógica del sistema de monedas está centralizada en el GameManager, que actúa como controlador global. Este se encarga de registrar el número total de monedas existentes en el nivel, llevar la cuenta de cuántas monedas ha recogido el jugador y comprobar de forma constante si se ha alcanzado el número requerido para permitir la victoria. Centralizar esta lógica evitó duplicaciones de código y redujo errores, asegurando que todas las comprobaciones relacionadas con el progreso del jugador se realizan desde un único punto del sistema.

Cada moneda dispone de un collider configurado como Trigger, lo que permite detectar el contacto con el jugador sin interferir en el movimiento. Cuando el jugador entra en contacto con una moneda, se ejecuta la lógica de recogida: se incrementa el contador global de monedas en el GameManager y la moneda se destruye inmediatamente. Esta desactivación es esencial para evitar que una misma moneda pueda ser recogida más de una vez y garantiza la coherencia del sistema.

La WinZone permanece bloqueada mientras no se hayan recogido todas las monedas requeridas. Solo cuando el GameManager detecta que el número de monedas recogidas coincide exactamente con el total definido para la dificultad seleccionada, se habilita la zona de victoria. A partir de ese momento, el jugador puede acceder a ella y finalizar la partida. Esta restricción asegura que las monedas sean un requisito indispensable para ganar, reforzando la exploración del escenario y aumentando la tensión durante el desarrollo del juego.

Durante la implementación del sistema de monedas surgieron numerosos problemas que obligaron a revisar tanto la lógica de programación como la configuración de los objetos en Unity. Uno de los primeros errores detectados fue que las monedas no aparecían en la escena. Este problema se debió principalmente a prefabs mal configurados, monedas colocadas fuera del área visible o a una altura incorrecta en el eje Y, así como a objetos que permanecían desactivados por error en la jerarquía. Como consecuencia, el código funcionaba correctamente, pero el jugador no podía ver ni recoger ninguna moneda.

Otro problema recurrente fue que las monedas no se recogían al pasar por encima de ellas. En estos casos, el jugador atravesaba visualmente la moneda sin que se activara la recogida. Las causas principales fueron colliders que no estaban marcados como Trigger, capas del jugador mal configuradas o errores en la comparación de etiquetas utilizadas para identificar al jugador. Estos fallos evidenciaron la importancia de una correcta configuración de colisiones y capas en Unity.

También se detectaron situaciones en las que una misma moneda se contabilizaba varias veces. Este error se producía cuando la moneda no se desactivaba tras la recogida o cuando el método de detección de colisión se ejecutaba varias veces durante un mismo contacto. La

solución consistió en desactivar o destruir la moneda de forma inmediata al ser recogida, evitando así múltiples incrementos del contador.

Finalmente, uno de los problemas más críticos fue que la WinZone se activaba antes de tiempo, permitiendo al jugador ganar la partida sin haber recogido todas las monedas. Este fallo se originaba por comparaciones incorrectas entre el número de monedas recogidas y el total requerido, así como por una inicialización errónea de la variable `totalCoins`. La solución pasó por revisar la lógica de comprobación y centralizar toda la validación en el `GameManager`, garantizando que la condición de victoria solo se cumple cuando realmente se han recogido todas las monedas necesarias.

En conjunto, el sistema de monedas evolucionó desde una idea inexistente en el diseño inicial hasta convertirse en uno de los pilares fundamentales del juego. Su implementación no solo aumentó la dificultad y el interés de la experiencia, sino que también obligó a mejorar la arquitectura del proyecto y a reforzar la coherencia entre lógica, configuración y diseño del nivel.

7. WinZone, estados finales y bucle de juego

La WinZone actúa como el elemento que desencadena el final de la partida y sirve como punto de transición entre el desarrollo del juego y su resolución. Su función no es introducir nuevas mecánicas, sino cerrar el ciclo de juego una vez que el jugador ha cumplido todos los objetivos establecidos previamente. De este modo, la WinZone se integra dentro del flujo general del juego como un mecanismo de finalización controlado y coherente con el estado global de la partida.

Uno de los problemas más importantes detectados durante el desarrollo fue que, en versiones iniciales, la victoria se activaba automáticamente al iniciar la partida. Este error evidenció que el sistema de estados no estaba correctamente sincronizado y que las comprobaciones finales se ejecutaban en momentos inadecuados del ciclo del juego. Para solucionarlo, fue necesario revisar el orden de inicialización y asegurar que las condiciones de victoria y derrota solo se evalúan cuando el juego se encuentra realmente en el estado de partida activa, evitando así transiciones prematuras o incoherentes.

Una vez corregido este comportamiento, se definió claramente el funcionamiento de los estados finales del juego. Cuando el jugador gana la partida, el sistema cambia al estado de victoria, se muestra la interfaz correspondiente y se bloquean las acciones de juego. De forma equivalente, cuando el jugador pierde, el juego pasa al estado de derrota, deteniendo igualmente la jugabilidad y mostrando la información adecuada. En ambos casos, el sistema está preparado para reiniciar la partida, devolviendo el juego a su estado inicial.

Este diseño permite establecer un **bucle de juego completo**, compuesto por inicio, desarrollo, finalización y reinicio. Gracias a este loop, el jugador puede repetir la experiencia de manera continua, ya sea para mejorar su rendimiento, probar diferentes dificultades o explorar nuevas estrategias. La existencia de este ciclo cerrado garantiza que el juego no quede en estados intermedios inconsistentes y refuerza la estabilidad general del proyecto.

En conjunto, la gestión de la WinZone y de los estados finales no solo resuelve el cierre de la partida, sino que define claramente la estructura global del juego. Este enfoque aporta coherencia al diseño, facilita el mantenimiento del código y asegura una experiencia de usuario controlada y predecible, alineada con los objetivos planteados desde el inicio del proyecto.

8. GameManager y Control del juego

El GameManager se consolidó como el núcleo del control del videojuego y como uno de los elementos más relevantes de la arquitectura final del proyecto. Su función principal es coordinar el estado global del juego y garantizar que todos los sistemas dependientes funcionen de forma sincronizada. A través de esta clase se gestiona el ciclo completo de la partida, incluyendo los estados de menú, juego activo, victoria y derrota, así como la activación y desactivación de la interfaz de usuario asociada a cada uno de ellos.

Además de la gestión de estados, el GameManager centraliza la lógica relacionada con el progreso del jugador, especialmente el conteo de monedas recogidas y la activación de la WinZone una vez cumplidos los requisitos necesarios para ganar la partida. También controla la activación de la alerta global provocada por determinados tipos de enemigos, limitándola en el tiempo para evitar situaciones injustas o desequilibradas. Esta centralización permitió reducir el acoplamiento entre scripts y evitar que múltiples clases intentasen modificar el estado del juego de forma independiente.

Durante el desarrollo surgieron numerosos problemas relacionados con este sistema. En fases iniciales, el juego entraba directamente en estado de derrota nada más comenzar la partida, o mostraba las pantallas de victoria y derrota de forma incorrecta. También se detectaron fallos en el funcionamiento del botón de reinicio, que en algunos casos no devolvía el juego a su estado inicial. Todos estos errores estaban relacionados con una mala sincronización entre el estado del juego y la activación de la interfaz gráfica. La solución pasó por revisar el orden de inicialización, reforzar las comprobaciones de estado y asegurar que cualquier cambio global se realizase exclusivamente a través del GameManager, consolidándolo como la única fuente de verdad del sistema.

9. Comparación entre el diseño inicial y el diseño final

El diseño inicial del proyecto planteaba un videojuego sencillo, ambientado en un laboratorio cerrado, con enemigos de comportamiento básico. La gestión del juego estaba repartida entre varios scripts, lo que hacía el sistema difícil de mantener y propenso a errores cuando se introducían cambios.

El diseño final difiere notablemente de esta idea original. El escenario pasó a ser un castillo medieval abierto, lo que obligó a replantear la navegación y el comportamiento de los enemigos. La inteligencia artificial evolucionó hacia un sistema basado en estados claramente definidos, capaz de reaccionar de forma coherente ante la presencia del jugador. La navegación se refinó mediante el uso de un NavMesh ajustado y un sistema propio de

pathfinding con A*, que permitió un mayor control sobre las rutas y evitó comportamientos erráticos.

Asimismo, la gestión del juego dejó de estar dispersa para centralizarse completamente en el GameManager, lo que mejoró la estabilidad, facilitó la depuración y permitió implementar un bucle de juego completo y controlado. Esta comparación pone de manifiesto cómo el proyecto evolucionó desde una idea simple hacia una solución más compleja y robusta, adaptándose a los problemas reales surgidos durante el desarrollo.

Con el objetivo de aumentar la credibilidad y la inmersión del juego, se incorporaron distintos elementos que refuerzan la sensación de estar ante un sistema vivo y reactivo. Se añadió música de fondo para acompañar la experiencia de juego y generar tensión durante la exploración del escenario, contribuyendo a una mayor implicación del jugador. Asimismo, se trabajó especialmente el movimiento del personaje principal para que resultara natural y reconocible, permitiendo desplazamientos fluidos tanto hacia delante como hacia atrás, evitando movimientos rígidos o artificiales. En paralelo, el comportamiento visual y funcional de los enemigos se diseñó para que transmitieran la sensación de drones de vigilancia, capaces de patrullar el entorno y reaccionar activamente ante la presencia del jugador, reforzando la idea de persecución y peligro constante. En conjunto, estos elementos no solo mejoran el apartado estético y sonoro del juego, sino que contribuyen de forma directa a una experiencia más creíble, coherente y envolvente.

10. Conclusiones finales

El desarrollo del proyecto fue un proceso progresivo y no lineal, marcado por numerosos errores, pruebas fallidas y refactorizaciones constantes. Lejos de ser un inconveniente, estos problemas se convirtieron en una parte esencial del aprendizaje, ya que obligaron a replantear decisiones, mejorar la arquitectura del código y aplicar de forma práctica los paradigmas de programación estudiados en la asignatura.

El resultado final es un videojuego funcional, coherente y bien estructurado, que cumple con los objetivos planteados inicialmente y demuestra una correcta aplicación de conceptos como la programación orientada a objetos, la gestión de estados, los patrones de diseño y la inteligencia artificial. Más allá del producto final, el valor principal del proyecto reside en el proceso de desarrollo, en la capacidad de adaptación ante los problemas y en la toma de decisiones técnicas fundamentadas, aspectos clave en cualquier proyecto de programación de mayor envergadura.

11. Uso de la IA:

Durante el desarrollo del proyecto se utilizó la ayuda de herramientas de inteligencia artificial como apoyo puntual en el proceso de aprendizaje y resolución de problemas. Esta asistencia se empleó principalmente para aclarar conceptos teóricos, comprender mejor determinados errores de programación, revisar enfoques de diseño y obtener orientación en la estructuración del código y de la documentación. En ningún caso la inteligencia artificial sustituyó el trabajo práctico realizado, sino que actuó como una guía complementaria.